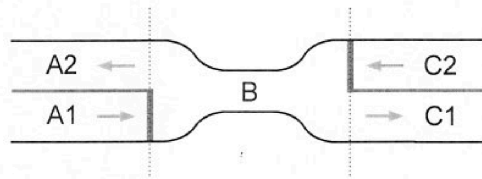


OS_2023-01-24

1) Synchronisation mit Semaphoren (30)

Der Fahrzeugverkehr an einer Straßenverengung (siehe Skizze) wird mit Hilfe von Prozessen simuliert. Die skizzierte Straße können Sie sich nach links und rechts beliebig verlängert vorstellen.



Das Codestück `car_left2right` simuliert die Fahrt eines von links, auf dem Fahrstreifen A1 kommenden Autos, das die Engstelle B passiert und dann rechts, auf Fahrstreifen C1 weiterfährt. Dabei stehen die Funktionsaufrufe `A1()`, `B()` und `C1()` für das Durchfahren der entsprechenden Straßenstücke. Analoges gilt für `car_right2left` und die Aufrufe von `C2()`, `B()` und `A2()`.

Ergänzen Sie die grau hinterlegten Felder der Codestücke mit Initialisierungscode bzw. mit Code zur Synchronisation der Fahrzeuge. Die Synchronisation der Fahrzeuge muss mittels *Semaphoren* erfolgen und folgende Anforderungen erfüllen:

- Im Bereich der Fahrbahnverengung (Abschnitt B) dürfen sich zu jedem Zeitpunkt nur Autos befinden, die in die selbe Richtung fahren.
- Im Abschnitt B dürfen sich maximal 3 Fahrzeuge gleichzeitig befinden.
- Befinden sich bereits Fahrzeuge im Abschnitt B, so werden neu ankommende Fahrzeuge, die Abschnitt B in die gleiche Richtung durchfahren wollen, wie die bereits in B befindlichen Fahrzeuge, bevorzugt gegenüber den Fahrzeugen, die in die Gegenrichtung fahren möchten.

Hinweis: Überlegen Sie, welche Teile der in der Vorlesung präsentierten Lösungen zu Synchronisationsaufgaben für die Lösung des hier gegebenen Synchronisationsproblems verwendbar sind.

Initialisierungen:

```

init(L, 1);
init(R, 1);

init(limit, 3);
init(bridge, 1);

int countL = 0;
int countR = 0;

```

void car_left2right(void)

{

A1();

```

wait(L);
countL++;

```

```

if (countL == 1) {
    wait(bridge);
}

```

```

signal(L);
wait(limit);

```

B();

```

signal(limit);
wait(L);
countL--;

```

```

if (countL == 0) {
    signal(bridge);
}
signal(L)

```

C1();

}

void car_right2left(void)

{

C2();

```

wait(R);
countR++;

```

```

if (countR == 1) {
    wait(bridge);
}

```

```

signal(R);
wait(limit);

```

B();

```

signal(limit);
wait(R);
countR--;

```

```

if (countR == 0) {
    signal(bridge);
}
signal(R);

```

A2();

}

2) Page Replacement (25)

Gegeben ist ein Arbeitsspeicher mit vier Frames, dessen Seiten mit unterschiedlichen Ersetzungsstrategien ersetzt werden sollen: mit OPT, LRU, bzw. mit dem Clock Algorithmus. Ergänzen Sie in den Tabellen für jeden Algorithmus die Speicherinhalte für jeden Frame nach jedem Zugriff der angegebenen Seitenzugriffsfolge an. Die Seitenzugriffsfolge ist für alle Algorithmen gleich. Sie ist jeweils in der Kopfzeile der Tabelle gegeben. Geben Sie in den Spalten die Speicherbelegung nach dem entsprechenden Seitenzugriff an und kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Der Arbeitsspeicher ist am Beginn leer. Für die ersten vier Speicherzugriffe sind die Tabellen bereits ausgefüllt.

OPT-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
1		B	B	B	B	B	B	B	B	F	F	F	F	F	F	F	F
2			C	C	C	E	E	E	E	E	E	E	E	E	E	E	E
3				D	D	D	D	D	D	D	D	D	D	D	D	D	D
PF	X	X	X	X		X				X							

LRU-Strategie:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A	E	E	E	E	F	F	F	F	F	F	F	F
1		B	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D
2			C	C	C	C	C	C	A	A	A	A	A	A	A	A	A
3				D	D	D	D	D	D	D	E	E	E	E	E	E	E
PF	X	X	X	X		X			X	X	X	X					

Clock-Algorithmus:

	A	B	C	D	C	E	D	B	A	F	E	D	A	F	D	A	E
0	A	A	A	A	A ₁	E ₁	E ₁	E ₁	E ₁	E ₀	E ₁	E ₁	E ₁	E ₁	E ₁	E ₁	E ₁
1		B	B	B	B ₁	D ₀	D ₀	D ₁	B ₀	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁	F ₁
2			C	C	C ₁	C ₀	C ₀	C ₀	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁	A ₁
3				D	D ₁	D ₀	D ₁	D ₁	D ₁	D ₀	D ₀	D ₁	D ₁	D ₁	D ₁	D ₁	D ₁
PF	X	X	X	X		X			X	X							

Vergleichen und diskutieren Sie die Anzahl der bei den Seitenersetzungsstrategien beobachteten Page Faults. Entspricht das Ergebnis dem zu erwartenden Verhalten?

Natürlich ist zu erwarten, dass OPT die beste Lösung liefert, da die Lösungsstrategie rein hypothetisch ist und immer das beste Ergebnis ausspuckt.

LRU schneidet hier am schlechtesten ab, weil es nur auf die Vergangenheit blickt und bei der Zugriffsfolge, Seiten die gerade erst verdrängt wurden direkt wieder angefordert.

Clock sollte eigentlich nur eine effiziente Annäherung an LRU sein, allerdings schneidet er hier unerwartet besser ab.

3) Fragen zu Betriebssystemen (45)

Nennen sie zwei Abstraktionen, die ein Betriebssystem dem Benutzer bietet. Welche Aufgaben führt das Betriebssystem durch, um diese Abstraktionen zu realisieren? (4)

- Ungestörte Programmabarbeitung -> Prozessmanagement, Deadlocks, Memorymanagement
- "private" Maschine -> Betriebssystem ist für Zugriffsschutz und Datensicherheit zuständig.

Welcher Vorteil ergibt sich aus der Einführung von *Threads* für den Benutzer? Wodurch kommt es zu diesem Vorteil? Worauf muss der Benutzer bei der Programmierung von Threads achten? (3)

Dass man Programmteile parallel ablaufen lassen kann. Der Benutzer muss darauf achten, dass sich Threads einen gemeinsamen Speicher teilen.

Mit welcher logischen Struktur des I/O Systems versucht man, bei der Realisierung von I/O Funktionen sowohl ein einheitliches Programmierinterface, als auch eine möglichst gerätespezifische Ansteuerung zu erreichen? (4)

Mit der Schichtstruktur, aufgeteilt in:

- Logische I/O
- Geräte I/O
- und Scheduling und Control

Für die Lösung des Problems des geregelten Eintritts in einen kritischen Abschnitt werden drei Eigenschaften gefordert. (a) Nennen Sie diese drei Eigenschaften und erklären Sie deren Bedeutung. (b) Wodurch werden die drei Eigenschaften gewährleistet, wenn Semaphore zum Schutz eines kritischen Abschnitts verwendet werden? (5)

a)

- Mutual Exclusion: nur ein Prozess darf in den kritischen Abschnitt
- Progress: jeder Prozess muss irgendwann hinein und das darf nicht verzögert werden
- Bounded waiting: Nach request für kritischen Abschnitt gibt es nur eine limitierte Anzahl von Wartenden vor einem

b)

- Semaphore bestehen aus Value und einer Queue.
- Überprüft wird ob $value \leq 0$ oder $value > 0$ ist, je nach dem darf man rein, oder kommt in die Queue.
- Queue -> Progress + Bounded waiting
- Sobald man eintritt wird $value$ um 1 verringert -> Mutual Exclusion

Nennen Sie die Bedingungen für das Eintreten eines Deadlocks und erklären Sie diese. (5)

1. Mutual exclusion -> ausschließen von Prozessen (Einschränkung wie viele in einen Abschnitt dürfen)
2. Hold and wait -> Prozess kann Ressourcen halten während er auf andere wartet
3. no preemption -> man kann den Prozess im kA nicht terminieren / die Ressourcen nicht wegnehmen
4. circular waiting -> ein Prozess wartet auf Abhängigkeit die ein anderer blockiert, der aber auch nur auf den einen wartet. -> gegenseitiges blockieren

Nennen Sie vier Designprinzipien für die Konstruktion von sicheren Systemen. Geben Sie für jedes Prinzip ein Beispiel an. (4)

1. Open-Design: keine Sicherheit durch besonders schwere Codes -> muss verständlich bleiben
2. Default Einstellungen: keine Berechtigungen -> BSP User soll default mäßig kein Admin haben
3. Least Privilege: so wenig Rechte wie möglich -> User soll nur das machen können, was er wirklich machen muss, alles andere sollte nicht funktionieren
4. Überprüfung der Berechtigungen -> Auch ein halbes Jahr später sollte geprüft werden ob jeder die selben Rechte hat die anfangs zugewiesen worden sind.

Beschreiben Sie die folgenden Strategien für das CPU Scheduling und vergleichen Sie deren Eigenschaften: *Round Robin*, *SRT*, *Highest Response Ratio Next* und *Feedback Scheduling*. (6)

Round Robin

- Es wird immer zwischen den offenen Prozessen hin und her gewitched und immer möglichst gleich lang ein Prozess bearbeitet bis zum nächsten gewitched wird
- Alle werden gleichmäßig behandelt
- Benachteiligung von IO intensiven Programmen die viel Zeit bei IO Operationen verschwenden

SRT

- Shortest remaining time
- Wenn ein kürzerer Prozess kommt als der aktive, wird gewitched
- Gute Behandlung von kurzen Prozessen
- Starvation von langen Prozessen möglich

Highest Response Ratio Next

- Ausgleich zwischen kurz bevorzugen und Starvation vermeiden...
- Mit Selection Function

$$R = \frac{w + s}{s}$$

wobei w... bisherige Wartezeit und s... geschätzte service time

- Dadurch werden kürzere bevorzugt und lange nach einer gewissen Wartezeit.

Feedback Scheduling

- Neue Prozesse starten mit hoher prio
- Wenn sie nicht fertig werden im Zeitfenster kommen sie in eine Queue mit hoher Priorität
- Wenn das 2. mal passiert in eine Queue mit weniger prio, und so weiter

Was versteht man unter *Virtual Memory Management*? Welche Mechanismen benötigt man zur Realisierung von *Virtual Memory Management*? Welche Vorteile bietet es? (5)

- unter virtual memory management versteht man dynamische Adressübersetzung: logische Adressen referenzieren VM, Adressübersetzung bei jeder Ausführung
- Benötigt ist:
 - das Aufteilen des Speichers in Pages und Segmente
 - erlaubt es nur für den Prozess wichtige Pages in den RAM zu laden
 - Segmenttabelle für Überprüfung
 - Falls angeforderte page nicht im Ram -> Page Fault -> reinladen aus Sekundärspeicher
 - Resident Set -> Teile des Prozesses die gerade im RAM sind
- Vorteil: Prozess kann mehr Speicher brauchen als RAM bietet

Erklären Sie die Begriffe *Access Control List* und *Capability List*. Wozu und wie werden diese verwendet? (3)

Access Control List

- Liste in der für das zugehörige file steht, wer was machen darf
- "Wer hat Zugriff auf dieses File?"

Capability List

- Jeder Prozess besitzt eine liste von "Tickets"
- Ticket erlaubt Zugriff auf Objekt
- Besitz davon ist Zugriffserlaubnis
- "Was darf dieser Prozess alles machen"

Beschreiben Sie das typische Layout einer Disk bzw. eines Filesystems. Erklären Sie kurz die einzelnen Komponenten. (3)

- Master boot record: kümmert sich beim booten darum, dass die aktive Partition ihren boot block die Kontrolle gibt
- partition table: hat die meta daten der Partitionen
- Partitionen:
 - boot block (führt code aus der dann den Kernel des OS lädt)
 - super block hat Metadaten der partition
 - free management: verwaltet Belegung der Daten

- i-nodes: Datenstruktur für die Metadaten
- root-dir: Einstiegspunkt für File tree
- files und directories

In welcher Art von Systemen ist der Einsatz von kryptographischen Verfahren zur Sicherung der Vertraulichkeit und Integrität von Daten notwendig? Was stellen kryptographische Verfahren in diesen Systemen sicher? (3)

in offenen Systemen wie Internet nötig, basiert auf dem Besitz von geheimen Schlüsseln.

Sicherstellung durch Verschlüsseln der Nachricht (Confidentiality) und Signieren der Nachricht (Integrity / Authencity)